

.clamp() method

[link](#)

clamp abstract method

`num clamp( num lowerLimit, num upperLimit )`

Returns this `num` clamped to be in the range `lowerLimit`-`upperLimit`.

The comparison is done using `compareTo` and therefore takes `-0.0` into account. This also implies that `double.NaN` is treated as the maximal double value.

The arguments `lowerLimit` and `upperLimit` must form a valid range where `lowerLimit.compareTo(upperLimit) <= 0`.

Example:

```
var result = 10.5.clamp(5, 10.0); // 10.0
result = 0.75.clamp(5, 10.0); // 5
result = (-10).clamp(-5, 5.0); // -5
result = (-0.0).clamp(-5, 5.0); // -0.0
```

Implementation

```
num clamp(num lowerLimit, num upperLimit);
```

[link](#)

1) clamp(minimum, maximum, valueToClamp)

1. **minimum** : Number – The minimum value
2. **maximum** : Number – The maximum value
3. **valueToClamp** : Number – The value that should be clamped between the first two values.

Returns: the clamped number

Example

```
// set the clamping range to between 0 and 100, and clamp 105
gsap.utils.clamp(0, 100, 105); // returns 100

// in the same range, clamp -50
gsap.utils.clamp(0, 100, -50); // returns 0

// and clamp 20
gsap.utils.clamp(0, 100, 20); // returns 20
```

Utility Methods > clamp

clamp

Clamps a number between a given minimum and maximum. If the number you provide is less than the minimum, it will return the minimum. If it's greater than the maximum, it returns the maximum. If it's between the minimum and maximum, it returns the number unchanged.

HTML CSS JS

Fork Rerun



50

Type a number above.
The value will be clamped between 30 & 70.

2) clamp(minimum, maximum)

1. **minimum** : Number – The minimum value

2. **maximum** : Number – The maximum value

Returns: a reusable function that accepts 1 parameter – a value to clamp

If you omit the **valueToClamp** (3rd) parameter, the utility method will return a **reusable function** that's ready to clamp any value according to the minimum/maximum values provided initially. In other words, the returned function remembers the minimum and maximum values so that it can very quickly and efficiently do the clamping later as many times as you want.

Example

```
// get a clamping function that will always clamp to a range between 0 and 100
var clamper = gsap.utils.clamp(0, 100); // notice we didn't provide a valueToClamp

// now we can reuse the function to clamp any values:
console.log(clamper(105)); // 100
console.log(clamper(-50)); // 0
console.log(clamper(20)); // 20
```